

Simulazione Esame di Laboratorio di Sistemi Operativi

Traccia: La Partita di Carte

Durata della prova: 180 minuti (3 ore)

Makefile

[3] Deve essere creato un Makefile di nome Makefile contenente un target prepara ed un target compila. Il target prepara deve creare un file avente come nome il contenuto della variabile FILE_DA_CREARE ricevuta in input da terminale. Il target compila deve compilare il file tavolo.c in un eseguibile chiamato tavolo.out solo dopo aver svolto il target prepara.

Programma in C

Un gioco di carte tra processi deve essere coordinato da un processo principale (**Tavolo**), che gestisce n processi figli (**Giocatori**) e si interfaccia con un processo esterno di supervisione (**Checker**). Il gioco è semplice: ogni giocatore ha 10 carte numerate e ad ogni mano gioca la propria carta più bassa. Vince la mano il giocatore con la carta più alta; vince il gioco chi ha vinto più mani.

Il codice deve essere strutturato in 3 file:

1. File `gioco.h`: già fornito, **NON va modificato**.
2. File `giocatore.c`: dovrà contenere il codice che verterà utilizzato per lanciare un giocatore. Va utilizzata la firma della funzione giocatore definita in `gioco.h`.
3. File `tavolo.c`: dovrà contenere la logica principale del programma (il main).

Avvio del Tavolo

[3] Il processo principale tavolo.out gestirà i vari giocatori e i vari eventi. Il programma deve essere lanciato accettando esattamente 2 parametri:

```
./tavolo.out <gameSettings> <checkerPID>
```

dove `<gameSettings>` è un file esistente nel sistema e `<checkerPID>` è il PID di un processo già esistente. Un numero diverso di parametri, un file non esistente o un PID non esistente devono far terminare il programma con **codice 40**. Il Tavolo dovrà rimanere attivo fino alla fine del programma.

Creazione dei Giocatori

[4] Il Tavolo deve leggere il file `<gameSettings>` contenente un numero di giocatori `<n>` (compreso tra 2 e 10) e deve quindi creare esattamente `<n>` processi figli, uno per ogni giocatore. Una volta creati, i processi devono rimanere attivi fino alla fine del programma.

Canale di Comunicazione

[3] Il Tavolo deve creare una **pipe anonima** con ognuno dei figli. Queste pipe verranno usate dai giocatori per inviare le proprie carte al Tavolo durante le mani di gioco.

Distribuzione delle Carte

[4] Ogni giocatore deve leggere le proprie carte da una coda di messaggi identificata tramite il file `<gameSettings>` e il numero `<n>`. La coda (già esistente al momento della valutazione) verrà riempita con messaggi aventi come **tipo** il PID del giocatore destinatario e come **payload** un array di 10 interi rappresentanti le carte. Le carte sono interi compresi tra 1 e 9, inviate **ordinate dalla più bassa alla più alta**. Ogni giocatore dovrà attendere il messaggio a lui indirizzato.

```
typedef struct {
    long mtype;      // PID del giocatore destinatario
    int  carte[10];  // carte ordinate
} mazzo_t;
```

Es: verrà inviato un messaggio di tipo 34 con le carte 1, 2, 4, 6, 8, 8, 8, 8, 8 che dovrà essere letto dal giocatore con PID 34, ed un messaggio di tipo 35 con payload 3, 4, 4, 4, 4, 4, 5, 5, 5 per il PID 35.

[3] Una volta letto il proprio messaggio, ogni giocatore dovrà inviare un segnale SIGRTMIN al processo <checkerPID>, avente come **payload** la propria **carta più alta** (un intero).

Es: il giocatore 34 dovrà inviare un segnale con payload 8, il giocatore 35 un segnale con payload 5.

Turno di Gioco

[4] Ogni giocatore, in ordine di PID, riceverà un segnale SIGUSR1 all'inizio del proprio turno. Alla ricezione di questo segnale, il giocatore sceglie la propria **carta più bassa** ancora disponibile e la invia al Tavolo tramite la pipe creata nella sezione precedente. Il Tavolo, ricevuta la carta, deve stampare su stdout il suo valore.

Es: il giocatore 34 invierà al tavolo la carta 1 ed il tavolo stamperà 1\n, il giocatore 35 la carta 3 ed il tavolo stamperà 3\n.

N.B.: una volta usata una carta, questa deve essere scartata!

[2] Ricevute le carte da tutti i giocatori, il Tavolo calcola il **vincitore della mano** basandosi sul valore della carta (vince la più alta; in caso di parità, vince il giocatore con il **PID maggiore**). Il Tavolo stampa su stderr:

<PIDVincitore> <carta>

Es: se ci sono 2 soli giocatori, 34 e 35, il primo round lo vincerà il giocatore 35 con la carta 3 ed il tavolo scriverà 35 3\n su stderr. Il terzo round, un pareggio, lo vincerà il giocatore 35 con la carta 4 per PID maggiore.

Alla fine del turno, verrà effettuato un nuovo invio di SIGUSR1 per proseguire il gioco.

[2] Implementare il turno di gioco affinché funzioni con i segnali SIGUSR1 inviati in **ordine casuale** ai vari giocatori (quindi non più necessariamente in ordine di PID).

Fine del Gioco

[4] Alla fine del gioco, quando tutte le carte sono state giocate, il giocatore che ha vinto più mani deve scrivere il proprio PID e il numero di vittorie sul file <gameSettings>, **sovrascrivendone il contenuto**, con il formato:

<PID>-<vittorie>

Es: il giocatore 34 scriverà 34-7 sul file.